Doc. No.:	WG21/P0387R0
Date:	2016-07-11
Author:	Hans-J. Boehm
Email:	hboehm@google.com
Target audience: SG1 (Concurrency)

P0387R0: Memory Model Issues for Concurrent Data Structures

Prior discussions of both concurrent queue and distributed counter proposals raised the issue that we do not have a clear consensus about what memory model guarantees should be provided by concurrent data structure libraries. This is an attempt to frame that discussion. The ideal outcome would be a set of principles to be followed by later proposals.

This builds on an earlier email discussion with JF Bastien, Lawrence Crowl, Doug Lea, and Paul McKenney.

The existing memory model compromise

When WG21 first started discussing memory model issues, we compromised on a "sequential consistency by default with control where needed" policy. A programmer who is not an expert in memory models can program to a model in which thread execution was simply interleaved: At each time step one of the available threads performs its next operation. This requires only that data races are avoided, and that no operations on atomics explicitly specify a `memory_order`.

Over the years, I believe we have intentionally relaxed this in only two ways:

- We've relaxed it in cases in which no reasonable code can tell the difference. For example `set_new_handler()`, `get_new_handler()`, and similar functions are not sequentially consistent. This means that if someone tries to implement Dekker's Algorithm by calling `set_new_handler(foo1); x = get_terminate_handler()` in one thread and calling `set_terminate_handler(foo2); y = get_new_handler()` in another, and entering the critical section if x != foo1 or y != foo2 respectively, in fact both threads may see the original values, and hence both enter the critical section at the same time. The general feeling was that this doesn't matter, because this code is ridiculous enough that we don't need to support it.
- We have in a few cases disguised non-sequentially-consistent behavior as non-determinism. The `try_lock()` call is defined so that it can be treated as non-deterministically failing without reason, to hide the fact that `lock()` and `try_lock()` are not sequentially consistent, and it would be expensive and typically useless to change that.

We have not yet really had to address the issue for higher-level libraries.

Existing practice for concurrent data structure libraries

As far as I can tell, interfaces for existing concurrent data structure libraries are often unclear what they promise. Academic papers often consider only operations on a single data structure, and ignore the effect of those data structure operations on other objects. (In large part because they often assume sequential consistency, in which case the issue does not arise.)

In practice, at least some ordering properties are clearly crucial: If I initialize object in Thread 1, and pass it through a concurrent queue to Thread 2, those initializing stores in Thread 1 had better be visible to Thread 2. That means queue insertion and removal must at least establish a "synchronizes with" relationship between the insertion and removal of the corresponding item.

The specification of java.util.concurrent components is not entirely clear, but based on a conversation with Doug Lea, the intent is to provide just these semantics: An operation that visibly modifies a concurrent data structure synchronizes with an operation that observes the result of this modification. This basically corresponds to acquire-release semantics.

Although this has worked out reasonably well in practice, it is prone to some programmer surprises that do not fall into either of the exception categories from the previous section. Consider the following, where both `q` and `log` are concurrent queues:

```
Thread 1:                       Thread 2:                       Thread 3:
q.push(1);                      log.push("pushed 2");           q.pop() yields 2
log.push("pushed 1");           q.push(2);                      q.pop() yields 1
```

With only this kind of acquire/release ordering, there is nothing to prevent the log containing

```
pushed 1
pushed 2
```

indicating the opposite order of what the code observed. This does not appear beginner-friendly, and pretty clearly does not qualify under either of the existing relaxations of the original compromise.

Also note that this is largely distinct from the occasionally proposed relaxations of e.g. FIFO ordering requirements for a single queue. Here both `q` and `log` individually obey full FIFO ordering requirements. It's just that the combined behavior cannot be explained in terms of a simple thread interleaving.

Implementation costs

I do not believe we currently have a good understanding of the implementation cost of stronger memory ordering semantics.

Any otherwise correct implementation that internally uses only locks and sequentially consistent atomics will itself be sequentially consistent. For more complex data structures, and if lock-freedom is not semantically required, lock-based implementations are often very competitive, since implementations often involve fewer memory fences than lock-free alternatives. This is especially true on weakly-ordered architectures like ARM.

Conversely, it is often very difficult to implement an API that requires sequential consistency (i.e. data structure operations participate in the total order S of sequentially consistent operations from 29.3p3[atomics.order]) using non-sequentially consistent atomics. (This issue is studied in some recent academic papers. A good starting point may be Batty, Dodds, and Gotsman. "Library Abstraction for C/C++ Concurrency", POPL 2013)

Policy options

I propose that we adhere to existing policy and ensure that any visible sequential consistency violations are either

- explicitly flagged with `memory_order` arguments,
- unobservable by reasonable code,
- or disguised as non-determinism in the specification.

This raises the question of how explicitly weaker ordering should be flagged. There are two obvious options:

1. As we do now, by passing a `memory_order` argument to member functions.
2. By providing different variants of the data structure as a whole, e.g. by templatizing a concurrent container class on a `memory_order` argument.

In the latter case, we might use `memory_order_relaxed` to indicate that the library guarantees no synchronizes-with relationships, `memory_order_acq_rel` if it guarantees that when B observes the effects of A, then A synchronizes with B, and `memory_order_seq_cst` to indicate that interleaving semantics are preserved. `memory_order_acquire` and `memory_order_release` would not be useful in this context. (Potential `memory_order_consume` support is deferred to a future discussion.)

Clearly the former is better for a low-level library like `<atomic>`. But we also have some evidence that memory ordering for different operations is not always cleanly separable for higher-level operations. Normally `lock` and `unlock` operations preserve sequential consistency even if implemented as acquire-release operations, but the addition of an always-accurate `try_lock` changes that. (See Boehm, "Reordering Constraints for Pthread-Style Locks", PPoPP 2007.) In Posix, the addition of `sem_getvalue()` similarly potentially wreaks havoc with the implementation of the other operations.

I propose that we allow the second model for higher-level data structures like queues, and use the Technical Specification model to experiment with the trade-offs for particular library components.

If there is no expectation that a lock-free implementation would significantly improve performance, or would otherwise be useful, the specification should omit the `memory_order` parameter and simply guarantee sequential consistency.